

EDX_Project

PP

2026-07-05

Introduction

Overview

The purpose of this project is to develop a movie recommendation system for individual users, akin to Netflix challenge and seen in Harvard Course 8 Machine Learning.

Main method is based on machine learning based on the data made available by subset of Movielens, called edx. Key concepts shown in the course are explored which include movie recommendation system purely based on average rating and 5 features which are available through edx. The 5 features include “userId”, “movieId”, time data, “title” and “genres”. Finally, this project will measure the final outcome in the form of residual mean squared error (RMSE).

The assignment specifies the RMSE value of <0.86490 as full score worthy goal and this project shows how this could be achieved using 4 out of the 5 features.

The English (Australian) style was used throughout this documentation given author’s geographic background.

Basic data

The basic standardised training and test data are provided. Code is provided in rmd file and not visualised below for ease of viewing.

```
## Loading required package: tidyverse

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: caret

## Warning: package 'caret' was built under R version 4.4.1

## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
## lift
```

```
## Warning in set.seed(1, sample.kind = "Rounding"): non-uniform 'Rounding'
## sampler used

## Joining with `by = join_by(userId, movieId, rating, timestamp, title, genres)`
```

Method

Data cleaning

Tidy data has been already provided by Edx team and this step was not required.

Data exploration and basic visualisations

Basic understanding of the edx data is below:

```
#Basic information on the dataset
str(edx)
```

```
## 'data.frame': 9000055 obs. of 6 variables:
## $ userId : int 1 1 1 1 1 1 1 1 1 1 ...
## $ movieId : int 122 185 292 316 329 355 356 362 364 370 ...
## $ rating : num 5 5 5 5 5 5 5 5 5 5 ...
## $ timestamp: int 838985046 838983525 838983421 838983392 838983392 838984474 838983653 838984885 838983707 838984596 ...
## $ title : chr "Boomerang (1992)" "Net, The (1995)" "Outbreak (1995)" "Stargate (1994)" ...
## $ genres : chr "Comedy|Romance" "Action|Crime|Thriller" "Action|Drama|Sci-Fi|Thriller" "Action|Adventure|Sci-Fi|Thriller" ...
```

```
dim(edx) #number of rows and columns
```

```
## [1] 9000055 6
```

```
edx %>% slice(1:10) #example of the first 10 rows
```

```
##   userId movieId rating timestamp title
## 1      1      122      5 838985046 Boomerang (1992)
## 2      1      185      5 838983525 Net, The (1995)
## 4      1      292      5 838983421 Outbreak (1995)
## 5      1      316      5 838983392 Stargate (1994)
## 6      1      329      5 838983392 Star Trek: Generations (1994)
## 7      1      355      5 838984474 Flintstones, The (1994)
## 8      1      356      5 838983653 Forrest Gump (1994)
## 9      1      362      5 838984885 Jungle Book, The (1994)
## 10     1      364      5 838983707 Lion King, The (1994)
## 11     1      370      5 838984596 Naked Gun 33 1/3: The Final Insult (1994)
##                                     genres
## 1                                     Comedy|Romance
## 2                                     Action|Crime|Thriller
## 4                                     Action|Drama|Sci-Fi|Thriller
## 5                                     Action|Adventure|Sci-Fi
## 6                                     Action|Adventure|Drama|Sci-Fi
## 7                                     Children|Comedy|Fantasy
## 8                                     Comedy|Drama|Romance|War
## 9                                     Adventure|Children|Romance
## 10 Adventure|Animation|Children|Drama|Musical
## 11                                     Action|Comedy
```

```
names(edx) #names of the heading
```

```
## [1] "userId" "movieId" "rating" "timestamp" "title" "genres"
```

```

n_distinct(edx$movieId) #distinct number of movies

## [1] 10677

n_distinct(edx$userId) #distinct number of users

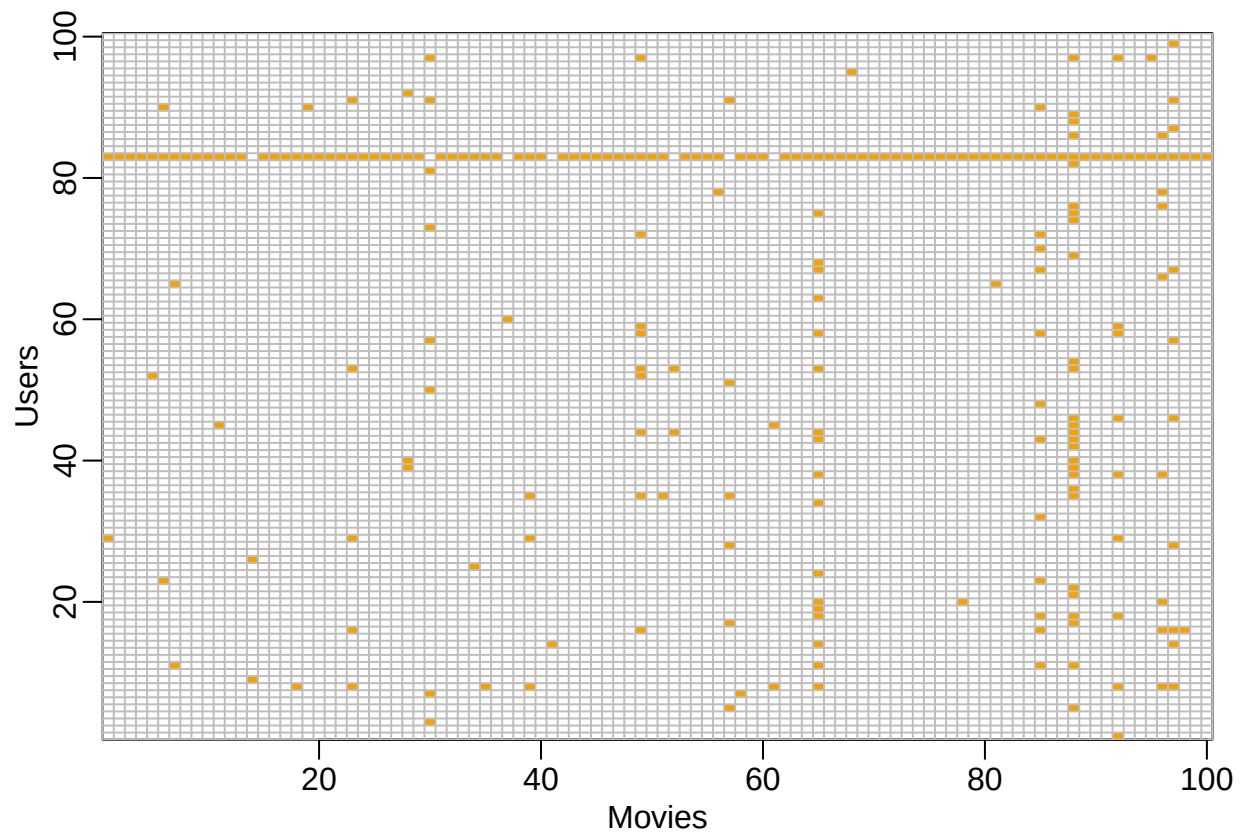
## [1] 69878

# The summary of ratings by each category
edx %>%
  select(rating) %>%
  group_by(rating) %>%
  summarise(frequencies = n())

## # A tibble: 10 x 2
##   rating frequencies
##   <dbl>         <int>
## 1  0.5           85374
## 2  1            345679
## 3  1.5          106426
## 4  2            711422
## 5  2.5          333010
## 6  3            2121240
## 7  3.5          791624
## 8  4            2588430
## 9  4.5          526736
## 10 5            1390114

# Sampling of 100 movies and 100 userId to check the extent of NA value - code is borrowed from Prof.
users <- sample(unique(edx$userId), 100)
rafalib::mypar()
edx %>% filter(userId %in% users) %>%
dplyr::select(userId, movieId, rating) %>%
mutate(rating = 1) %>%
pivot_wider(names_from = movieId, values_from = rating) %>%
(\(mat) mat[, sample(ncol(mat), 100)])() %>%
as.matrix() %>%
t() %>%
image(1:100, 1:100, ., xlab="Movies", ylab="Users")
abline(h=0:100+0.5, v=0:100+0.5, col = "grey")

```



```
# graphic representation of count of movies (graph_1) and those who rated (graph_2)
library(gridExtra)
```

```
## Warning: package 'gridExtra' was built under R version 4.4.1
```

```
##
```

```
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:dplyr':
```

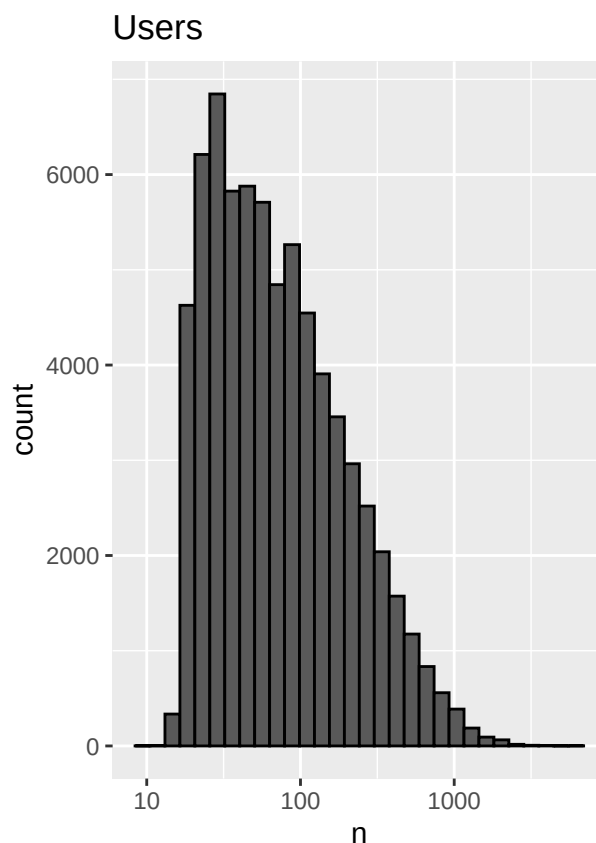
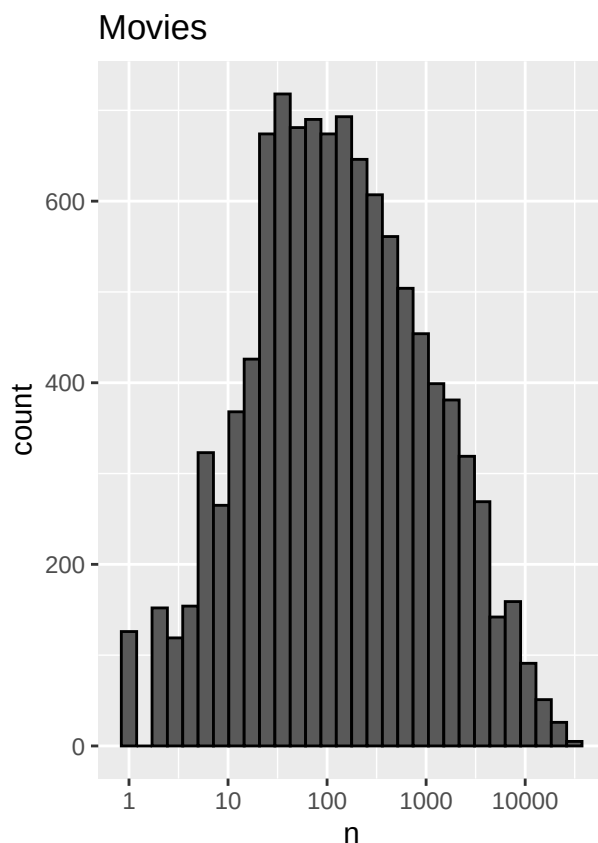
```
##
```

```
##      combine
```

```
graph_1 <- edx %>% count(movieId) %>%
  ggplot(aes(n)) +
  geom_histogram(bins = 30, color = "black") +
  scale_x_log10() +
  ggtitle("Movies")
```

```
graph_2 <- edx %>% count(userId) %>%
  ggplot(aes(n)) +
  geom_histogram(bins = 30, color = "black") +
  scale_x_log10() +
  ggtitle("Users")
```

```
grid.arrange(graph_1, graph_2, ncol = 2)
```



```
# numeric representation of count of movies (p1) and those who rated (p2)
edx %>% count(movieId) %>% summarise(mean = mean(n), min = min(n), Q1 = quantile(n, 0.25), median = median(n), Q3 = quantile(n, 0.75), max = max(n))

##           mean min  Q1 median  Q3   max
## 1 842.9386    1  30   122  565 31362

edx %>% count(userId) %>% summarise(mean = mean(n), min = min(n), Q1 = quantile(n, 0.25), median = median(n), Q3 = quantile(n, 0.75), max = max(n))

##           mean min  Q1 median  Q3   max
## 1 128.7967   10  32    62  141 6616
```

Above information tells us that 1. Data in the edx has 9000055 rows and 6 columns; 10677 distinct movies and 69878 raters. 2. Rating is our dependent variable of interest. We have 5 other variables which may act as our predictors, and these could be explored to identify confounding/ correlation. 3. Most of the ratings are whole numbers (e.g. 3, 4, 5) and that there is no 0. 4. Similar to Movielens dataset, there are many NAs for which we need to create recommendation work 5. All movies had rating (including just 1 rating); all users rated ≥ 10 times

RMSE and mathematical modeling approach

RMSE RMSE is residual mean squared error. We define $y_{u,i}$ as the rating for movie i by user u and denote our prediction $\hat{y}_{u,i}$.

The RMSE is then defined as following: $RMSE = \sqrt{\frac{1}{n} \sum_{u,i} (y_{u,i} - \hat{y}_{u,i})^2}$

RMSE can be interpreted similarly to a standard deviation. If this number is larger than 1, it means our typical error is larger than 1 star.

Mathematical modeling approach First model is based on simplest idea for the recommendation system. We can estimate this based on overall movie averages, which will be called μ . Hence same rating for all movies and all users would be made. This mathematically looks like this:

$Y_{u,i} = \mu + \epsilon_{u,i}$ where $\epsilon_{u,i}$ means errors sampled from the same distribution centered at 0. μ is the mean rating imputed for all movies.

Subsequent models will explore the impact of bias introduced by features mentioned above.

$$Y_{u,i} = \mu + b_i + b_u + b_t + b_g + \epsilon_{u,i}$$

where 1. b_i is bias for the movie effects, 2. b_u , user effects, 3. b_t , time effects, and 4. b_g , genre effects

Further machine learning was planned including matrix factorisation, and principal component analysis; however this was deemed not necessary to achieve the desired outcome.

Results

Model 1 - inputing average rating

Based on the above outlined mathematical modeling, first model looks at guessing the rating based on the average rating. Below tells us that RMSE of guessing the movie rating as an average of training set gives us RMSE of 1.061202.

```
# RMSE calculation function:
RMSE <- function(rating_Y, ratings_P){
  sqrt(mean((rating_Y - ratings_P)^2)) #Y= true, P=predicted
}

#A first model
mu <- mean(edx$rating) #this is mu_hat
mu #3.512465

## [1] 3.512465

model_1_rmse <- RMSE(final_holdout_test$rating, mu)
model_1_rmse

## [1] 1.061202

rmse_results <- tibble(Method = "Model 1. Average", RMSE = model_1_rmse)

knitr::kable(rmse_results)
```

Method	RMSE
Model 1. Average	1.061202

Model 2 - individual movie impact

Some movies are generally rated higher than others. We can supplement our previous model by adding the term b_i to represent average ranking for movie i . The combined model improves the RMSE to 0.9439087.

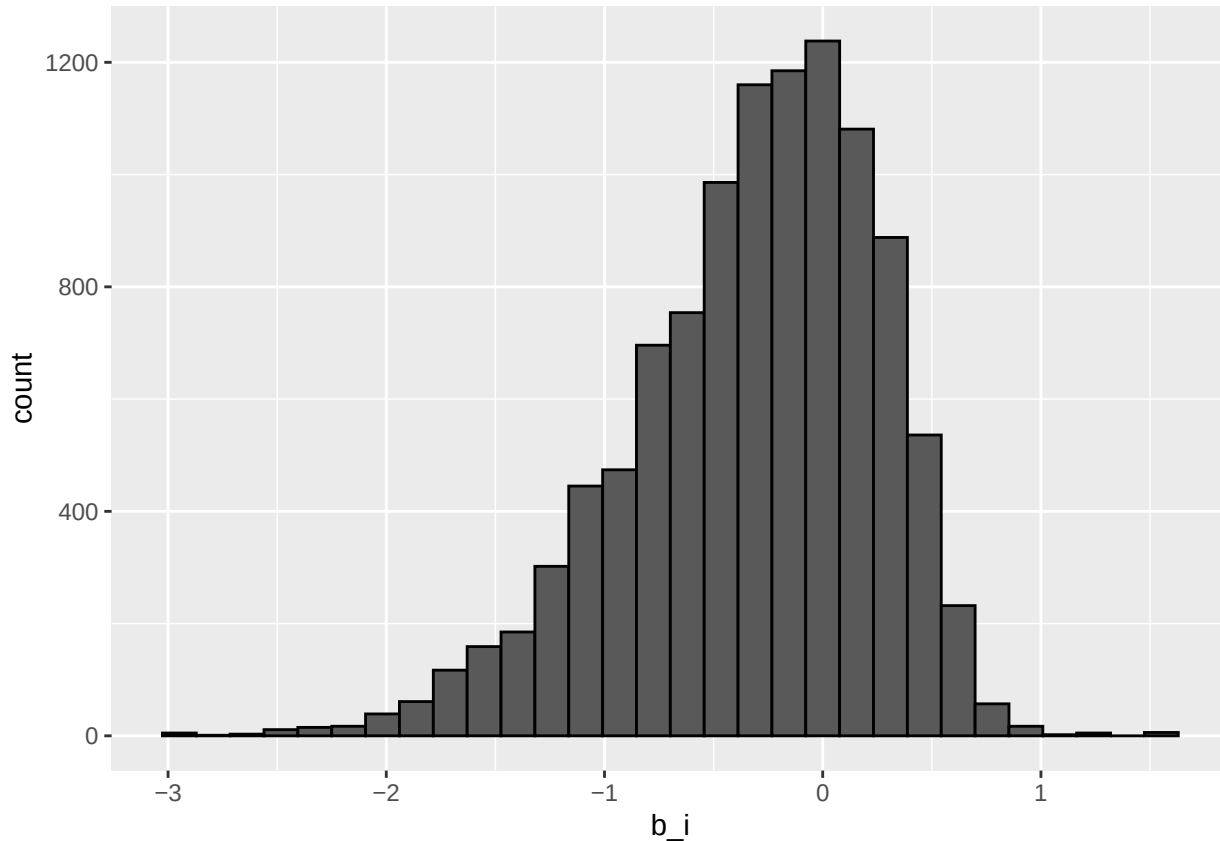
```
# mu is from earlier chunk
movie_mean <- edx %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))

#graph representation of the b_i (with 1.5 being the highest possible value given mean of 3.5, and -3.0)
edx %>%
```

```

group_by(movieId) %>%
  summarize(b_i = mean(rating - mu)) %>%
  ggplot(aes(b_i)) +
  geom_histogram(bins = 30, color = "black")

```



```

# additional impact of b_i on RMSE
rating_P <- mu + final_holdout_test %>%
  left_join(movie_mean, by='movieId') %>%
  pull(b_i)

model_2_rmse <- RMSE(rating_P, final_holdout_test$rating)
rmse_results <- bind_rows(rmse_results,
  tibble(Method="Model 2. Movie Effect Model",
    RMSE = model_2_rmse ))
knitr::kable(rmse_results)

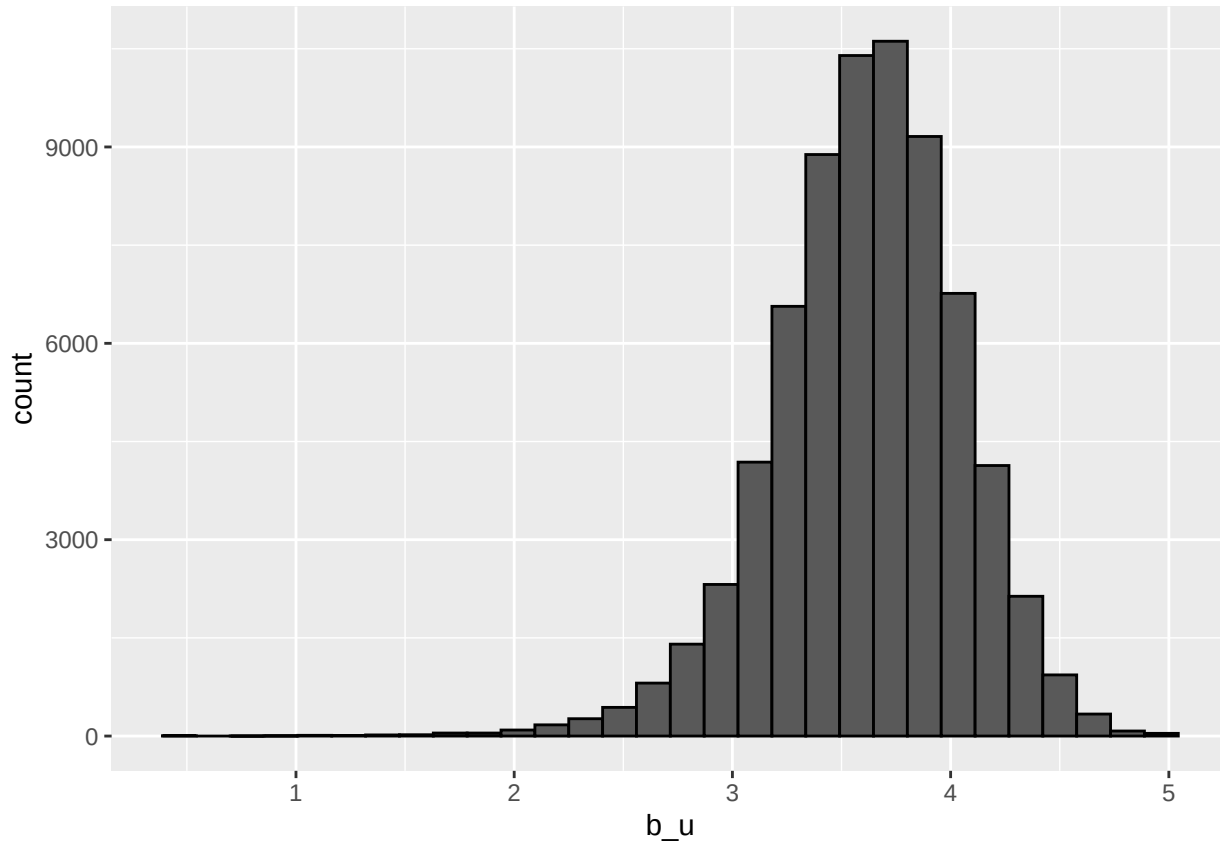
```

Method	RMSE
Model 1. Average	1.0612018
Model 2. Movie Effect Model	0.9439087

Model 3 - individual rater impact

Some raters rate movies higher than others. We can supplement our previous model by adding the term b_u to represent average ranking for users u . The combined model improves the RMSE to 0.8653488, which is a significant and massive improvement.

```
#individual rater average rating by counts
edx %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating)) %>%
  ggplot(aes(b_u)) +
  geom_histogram(bins = 30, color = "black")
```



```
# additional impact of b_u on RMSE
user_mean <- edx %>%
  left_join(movie_mean, by='movieId') %>%
  group_by(userId) %>%
  summarise(b_u = mean(rating - mu - b_i))

rating_P <- final_holdout_test %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%
  mutate(predicted = mu + b_i + b_u) %>%
  pull(predicted)

model_3_rmse <- RMSE(rating_P, final_holdout_test$rating)

rmse_results <- bind_rows(rmse_results,
  tibble(Method="Model 3. Movie + User Effects Model",
    RMSE = model_3_rmse ))

knitr::kable(rmse_results)
```


Method	RMSE
Model 1. Average	1.0612018
Model 2. Movie Effect Model	0.9439087
Model 3. Movie + User Effects Model	0.8653488

Due to a major drop in RMSE with b_u addition, it is worthwhile checking for any potential errors.

distribution of user effect appear reasonable

```
summary(user_mean$b_u)
```

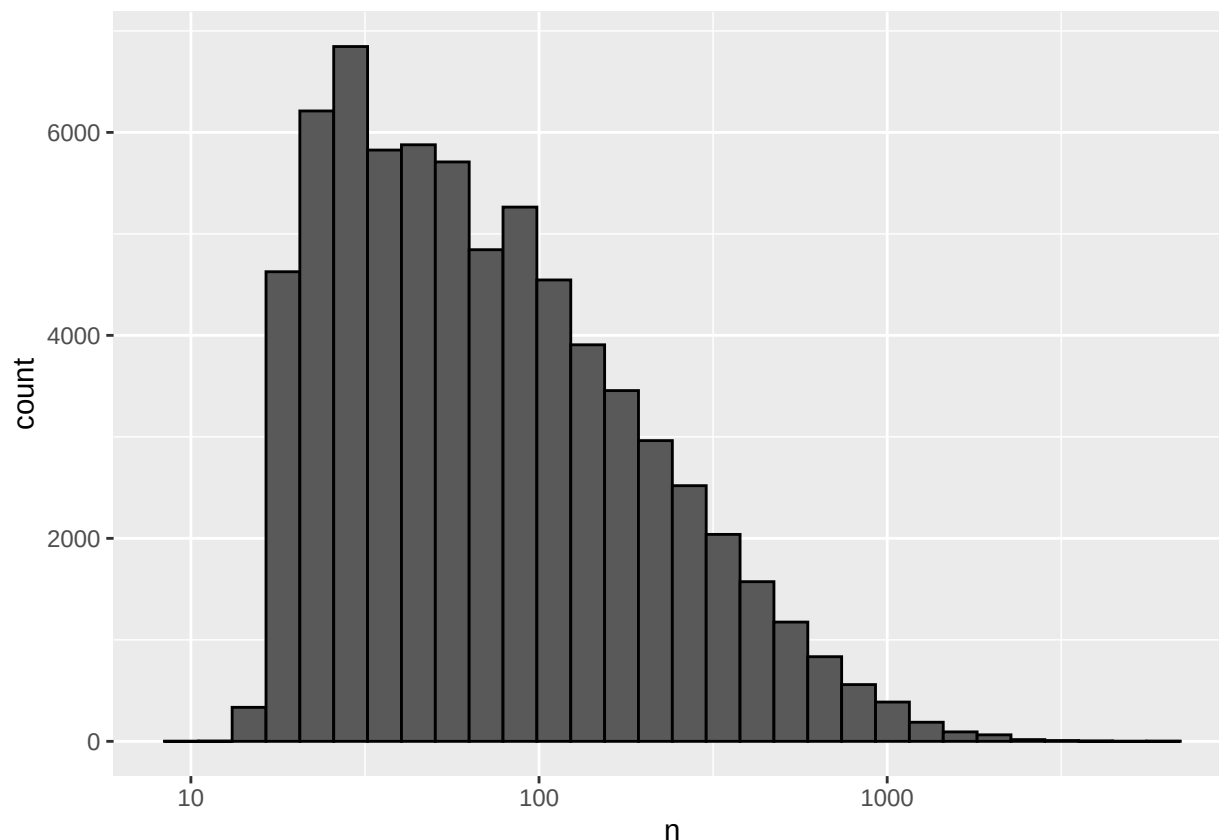
```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## -3.39056 -0.17948  0.07288  0.06134  0.32107  1.89056
```

number of ratings were shown previously and reproduced below. There are no raters with 1-3 rated movie

```
edx %>% count(userId) %>% summarise(mean = mean(n), min = min(n), Q1 = quantile(n, 0.25), median = median(n),
```

```
##      mean min Q1 median  Q3  max
## 1 128.7967  10  32    62 141 6616
```

```
edx %>% count(userId) %>% ggplot(aes(n)) +
  geom_histogram(bins = 30, color = "black") +
  scale_x_log10()
```



Below is testing for over-fitting by testing for RMSE on the train set. The result does not support m

```
predicted_train_ratings <- edx %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%
```

```

    mutate(predicted = mu + b_i + b_u) %>%
    pull(predicted)

train_rmse <- RMSE(predicted_train_ratings, edx$rating)

train_rmse

## [1] 0.8567039

```

Impact of time/date

Timing of movie release would impact on the rating. The relationship with time to movies are explored below, which shows that (1) relationship between time and rating is complicated. Overall rating decreased over the time of movie releases. In other words, more recent the movies, lower the rating. But (2) there was plateau of the rating decline by 2000, and there is up-trend in terms of rating since 2005.

```

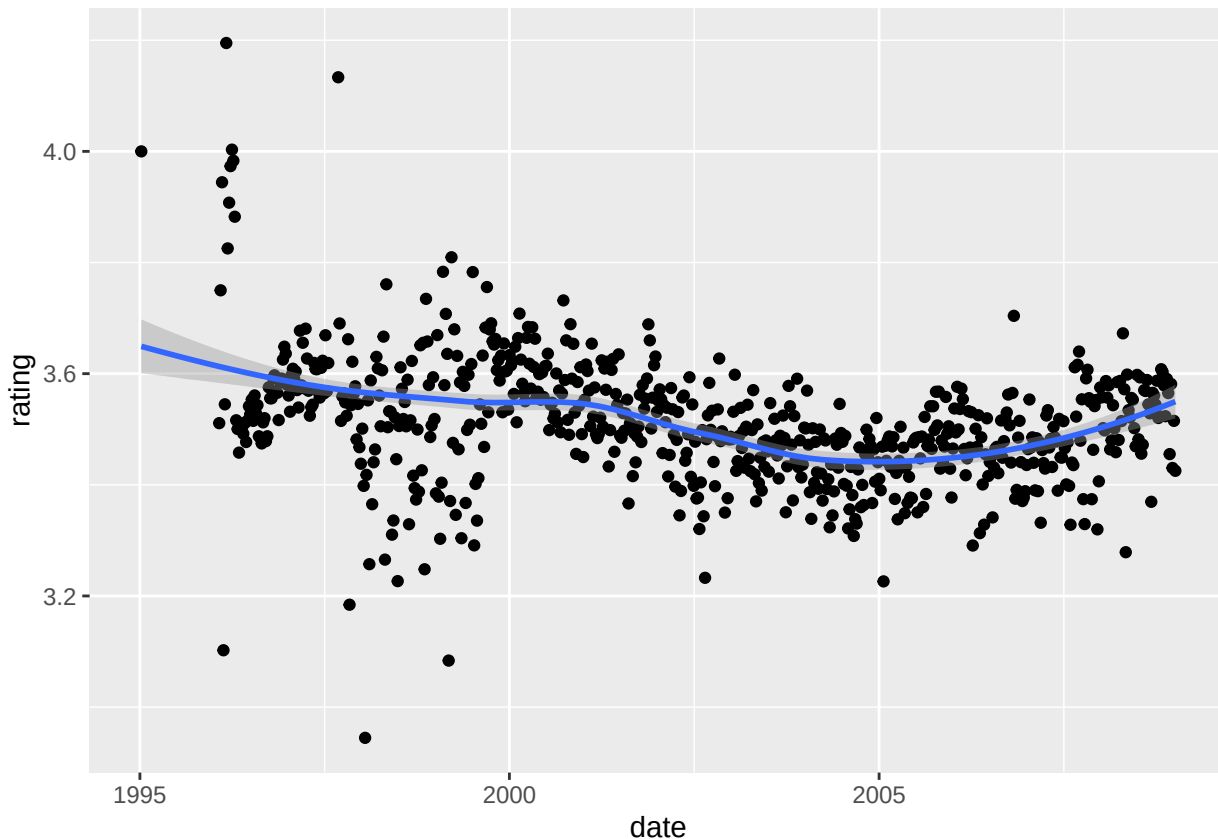
#create date column separated by "week"
edx <- edx %>%
  mutate(date = as_datetime(timestamp)) %>%
  mutate(date = floor_date(date, unit = "week")) #floor_date better than round_date for week specific e.

#similar operation is needed for the test_set
final_holdout_test <- final_holdout_test %>%
  mutate(date = as_datetime(timestamp)) %>%
  mutate(date = floor_date(date, unit = "week"))

#visualisation
edx %>% mutate(date = floor_date(date, unit = "week")) %>%
  group_by(date) %>%
  summarize(rating = mean(rating)) %>%
  ggplot(aes(date, rating)) +
  geom_point() +
  geom_smooth()

## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'

```



We can further supplement our previous model by adding the term b_t to represent average time impact for time t measured in weeks so that we can remove noise of individual date of movie release. The combined model improves the RMSE to 0.8652433 which is only a slight improvement.

#building the prediction model

```
date_mean <- edx %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%
  group_by(date) %>%
  summarise(b_t = mean(rating - mu - b_i - b_u))

rating_P <- final_holdout_test %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%
  left_join(date_mean, by='date') %>%
  mutate(predicted = mu + b_i + b_u + b_t) %>%
  pull(predicted)

model_4_rmse <- RMSE(rating_P, final_holdout_test$rating)

rmse_results <- bind_rows(rmse_results,
  tibble(Method="Model 4. Movie + User + Date Effects Model",
    RMSE = model_4_rmse ))

knitr::kable(rmse_results)
```

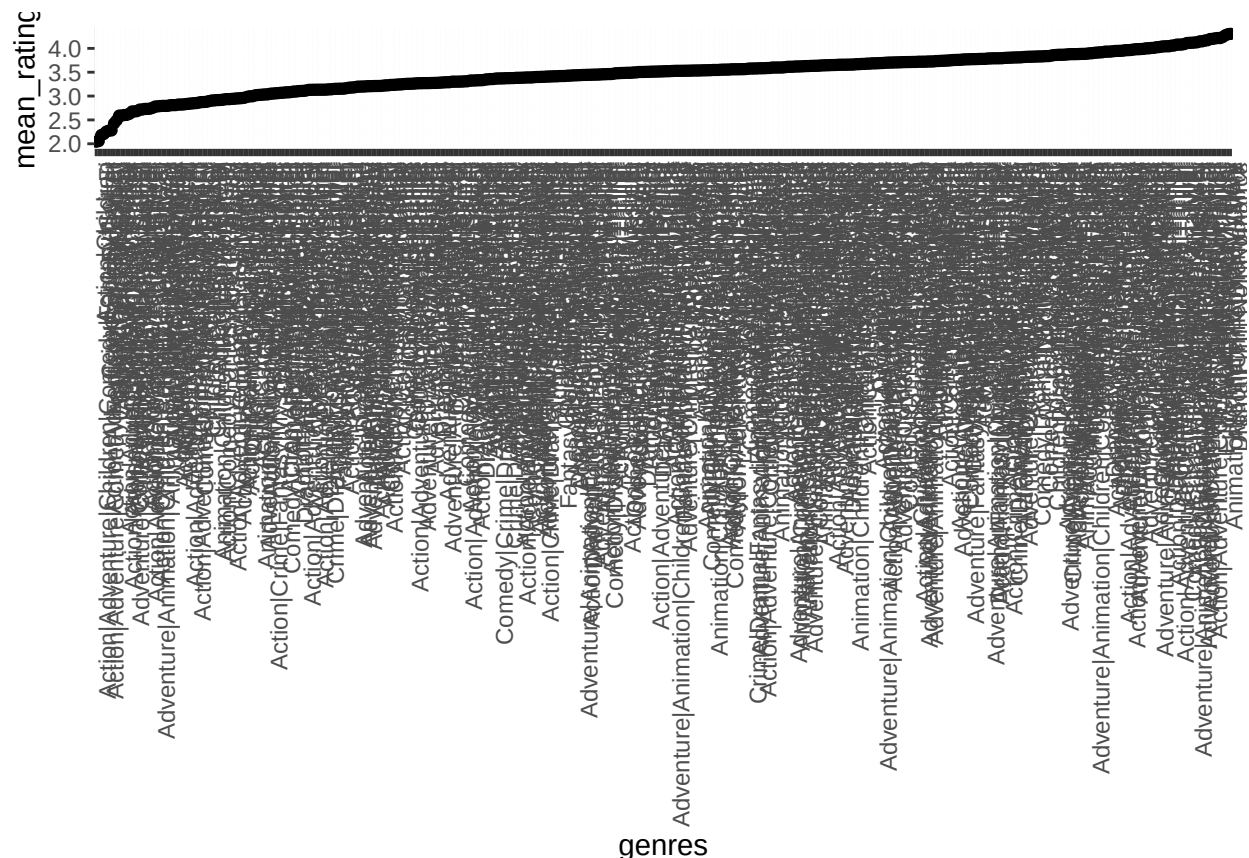
Method	RMSE
Model 1. Average	1.0612018
Model 2. Movie Effect Model	0.9439087
Model 3. Movie + User Effects Model	0.8653488
Model 4. Movie + User + Date Effects Model	0.8652433

Impact of genres

Genres are provided in the edx data set. Some genres are liked by users more than the others. This relationship with genres to movies are explored below.

#there is a clear effect of genres - please excuse the obscuration of the x-axis labels - this will be
edx %>%

```
group_by(genres) %>%
  summarize(mean_rating = mean(rating), n = n(), se = sd(rating)/sqrt(n), title=title[1]) %>%
  filter(n>=1000) %>%
  mutate(genres = reorder(genres, mean_rating)) %>% #arrange(mean_rating) this does not work
  ggplot(aes(x = genres, y = mean_rating, ymin = mean_rating - 2*se, ymax = mean_rating + 2*se)) +
    geom_point() +
    geom_errorbar() +
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



#10 lowest and maximum rated genres with ratings >1000:

```
edx %>% group_by(genres) %>%
  summarize(mean_rating = mean(rating), n = n(), se = sd(rating)/sqrt(n), title=title[1]) %>%
  filter(n >= 1000) %>%
```

```

mutate(genres = reorder(genres, mean_rating)) %>%
slice_min(n=10, genres)

## # A tibble: 10 x 5
##   genres                                mean_rating      n      se title
##   <fct>                                <dbl> <int> <dbl> <chr>
## 1 Action|Children                      2.04  3922 0.0174 3 Ni~
## 2 Action|Adventure|Children|Comedy|Fantasy|Sci-Fi 2.06  2832 0.0198 Supe~
## 3 Crime|Sci-Fi|Thriller                 2.18  2351 0.0205 Robo~
## 4 Action|Adventure|Fantasy|Thriller       2.21  4420 0.0160 Batm~
## 5 Action|Adventure|Comedy|Fantasy|Sci-Fi|Western 2.26  5301 0.0145 Wild~
## 6 Action|Children|Fantasy               2.28  2318 0.0230 Teen~
## 7 Children|Comedy|Sci-Fi                2.28  2940 0.0193 Hone~
## 8 Action|Comedy|Musical                 2.43  1305 0.0316 Blue~
## 9 Action|Crime|Fantasy                  2.49  2693 0.0204 Dare~
## 10 Fantasy|Horror|Thriller              2.59  1427 0.0306 Nigh~

edx %>% group_by(genres) %>%
  summarize(mean_rating = mean(rating), n = n(), se = sd(rating)/sqrt(n), title=title[1]) %>%
  filter(n >= 1000) %>%
  mutate(genres = reorder(genres, mean_rating)) %>%
  slice_max(n=10, genres)

## # A tibble: 10 x 5
##   genres                                mean_rating      n      se title
##   <fct>                                <dbl> <int> <dbl> <chr>
## 1 Drama|Film-Noir|Romance               4.30  2989 0.0145 Sunset Blv~
## 2 Action|Crime|Drama|IMAX               4.30  2353 0.0152 Dark Knigh~
## 3 Animation|Children|Comedy|Crime        4.28  7167 0.00962 Wallace & ~
## 4 Film-Noir|Mystery                     4.24  5988 0.0102 Maltese Fa~
## 5 Crime|Film-Noir|Mystery                4.22  4029 0.0120 Big Sleep,~
## 6 Film-Noir|Romance|Thriller              4.22  2453 0.0149 Notorious ~
## 7 Crime|Film-Noir|Thriller               4.21  4844 0.0119 Touch of E~
## 8 Crime|Mystery|Thriller                 4.20  26892 0.00515 Usual Susp~
## 9 Action|Adventure|Comedy|Fantasy|Romance 4.20  14809 0.00708 Princess B~
## 10 Crime|Thriller|War                    4.17  4595 0.0117 Manchurian~

```

We can further supplement our previous model by adding the term b_g to represent average genre impact for genre g measured. Rather than teasing out individual genre impacts (i.e. if there is combination of comedy and action, dissecting these out individually), genres as they are have been kept. The combined model improves the RMSE to 0.8648418 which is only a slight improvement but which achieves the RMSE that gets us the full mark.

```

#code for the model

genre_mean <- edx %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%
  left_join(date_mean, by='date') %>%
  group_by(genres) %>%
  summarise(b_g = mean(rating - mu - b_i - b_u - b_t))

rating_P <- final_holdout_test %>%
  left_join(movie_mean, by='movieId') %>%
  left_join(user_mean, by='userId') %>%

```

```

left_join(date_mean, by='date') %>%
left_join(genre_mean, by='genres') %>%
mutate(predicted = mu + b_i + b_u + b_t + b_g) %>%
pull(predicted)

model_5_rmse <- RMSE(rating_P, final_holdout_test$rating)

rmse_results <- bind_rows(rmse_results,
                          tibble(Method="Model 5. Movie + User + Date + Genre Effects Model",
                                RMSE = model_5_rmse ))

knitr::kable(rmse_results)

```

Method	RMSE
Model 1. Average	1.0612018
Model 2. Movie Effect Model	0.9439087
Model 3. Movie + User Effects Model	0.8653488
Model 4. Movie + User + Date Effects Model	0.8652433
Model 5. Movie + User + Date + Genre Effects Model	0.8648418

Conclusion

This project used RMSE as a measure of the loss function to see the impact of 4 out of 5 available features in edx data set. Using machine learning techniques, we were able to build movie recommendation system which can achieve RMSE of 0.8648418 stars. Although not perfect, the margin of error is within 1 star.

One major limitation is that more advanced machine learning techniques were not used in this instance. Partly this was due to limited time availability and the fact that Netflix team's winning RMSE of 0.8660 following extensive and advanced analysis using the Netflix data was slightly higher than the RMSE achieved with basic analysis performed in this project. Future projects could look at further use of advanced analysis on edx dataset.

In summary, use of basic machine learning methods achieved RMSE of 0.8648418 stars for edx dataset.

References

1. The backbone of the analysis was based on Prof Rafael A. Irizarry, available in the text book chapter 33 at <https://rafalab.dfci.harvard.edu/dsbook/>
2. Netflix challenge paper with a more detailed explanation was reviewed and available here: <https://www2.seas.gwu.edu/~simhaweb/champalg/cf/papers/KorenBellKor2009.pdf>
3. Large language model, ChatGPT, was used for checking for coding errors.